

Test-time Tensor Shape Contracts

A concise proposal and PR #748 implementation summary

What Jaxtyping adds

Jaxtyping extends Python type annotations with tensor *dtype*, *rank*, and *shape relationships*. It supports PyTorch tensors despite its name. A contract is executable only when paired with a runtime checker; otherwise it remains useful documentation.

```
from jaxtyping import Float, Int64
from torch import Tensor
```

```
def decode(q: Float[Tensor, "queries embedding_dim"],
           c: Float[Tensor, "candidates embedding_dim"]
           ) -> Float[Tensor, "queries candidates"]: ...
```

The repeated `embedding\_dim` binds at a call boundary: both inputs must have the same final dimension. This catches a common, expensive class of GNN interface bug near its source.

Shape vocabulary Axes are whitespace-separated. Exact integers are fixed, names bind equal sizes within one checked call, and dtype families such as `Float` allow any floating precision.

Contract	Meaning
<code>Float[Tensor, "batch 128"]</code>	Matrix with a fixed 128-wide final axis.
<code>Float[Tensor, "... channels"]</code>	Any leading rank, followed by <code>channels</code> . <code>...</code> is anonymous variable rank.
<code>Float[Tensor, "##batch features"]</code>	batch may equal its bound size <i>or 1</i> ; this enables broadcasting. It is not an optional scalar. A scalar is <code>"</code> .
<code>Float[Tensor, "{width}"]</code>	Exact size taken from a runtime argument named <code>width</code> .
<code>Float[Tensor, "{self.hidden}+1"]</code>	Exact size derived from instance configuration.
<code>Float[Tensor, "*batch channels"]</code>	Named variable number of leading axes. Only one <code>*name</code> or <code>...</code> is allowed per shape.

```
def add_bias(x: Float[Tensor, "##batch features"],
             bias: Float[Tensor, "##batch features"]
             ) -> Float[Tensor, "batch features"]: ...
```

```
def make_vector(width: int) -> Float[Tensor, "{width}"]: ...
```

```
def channel_last(x: Float[Tensor, "... channels"]) -> Float[Tensor, "... channels"]: ...
```

Benefits and limits

Benefits

- Contracts document the true boundary, not merely Tensor.
- Runtime checks validate dtype, rank, fixed axes, and named-axis equality.
- Reusing a name exposes semantic coupling in code review.
- Tests fail at the offending call, before malformed data propagates.

Limits

- Checks cover only exercised paths and annotated boundaries.
- Wrong or over-tight contracts create false test failures and maintenance.
- Dynamic keyed heterogeneous containers cannot express key-to-axis equality.
- Do not guess a tensor's rank or semantics simply to add an annotation.

Choosing a checker

Jaxtyping supplies tensor-aware types; another library invokes them at runtime.

Option	Strengths	Tradeoffs
Typeguard 2.x	Mature, straightforward decorator and import-hook integration. Jaxtyping explicitly documents the 2.x series. It is the PR's choice.	General runtime checking can encounter static-only GiGL annotations and forward references. The PR filters all non-Jaxtyping annotations before Typeguard sees them. Jaxtyping documents known issues with Typeguard 3 and 4.
Beartype	A popular alternative with the same decorator-shaped interface. It can be selected for <code>@jaxtyped</code> or an import hook.	Not selected here because the PR already has a working Typeguard 2.x path. Switching would add a dependency and behavior change without a demonstrated GiGL need.
Pyrefly	Fast static type checker and language server. Useful for broad Python typing feedback in editors and CI.	Different tool: it does not perform runtime tensor value checks. It requires adopting and maintaining a repository-wide static type-checking program, including third-party and dynamic PyTorch/PyG boundaries. That is valuable independently, but larger than this targeted test contract proposal.

How runtime checking works

Jaxtyping creates a dynamic context per checked function. The checker calls `isinstance` against a tensor type; Jaxtyping binds named axes while checking arguments and verifies returns against the same bindings.

```
from jaxtyping import Float, jaxtyped
from typeguard import typechecked

@jaxtyped(typechecker=typechecked)
def outer(x: Float[Tensor, "batch left"],
         y: Float[Tensor, "batch right"])
    ) -> Float[Tensor, "batch left right"]:
    return x[:, :, None] * y[:, None, :]
```

For broad coverage, Jaxtyping also provides an import hook that automatically instruments functions and dataclasses imported after installation.

```
from jaxtyping import install_import_hook
hook = install_import_hook(
    modules=("gigl", "examples"), typechecker="typeguard.typechecked"
)
# Later imports from those packages are instrumented.
```

Important Runtime checks are a complement to static typing, not a replacement. They validate actual values on executed test paths. Static checking validates a wider program model before execution, but cannot generally prove runtime tensor dimensions.

Why test-time only

Production training and inference avoid instrumentation overhead, import-order sensitivity, and a new user-facing operational mode. The contracts remain normal annotations in production. Tests provide the feedback loop where a precise failure is most useful.

PR #748 in GiGL

Targeted contracts at stable boundaries, enforced only by test launchers.

Implementation

Unit, integration, and E2E launchers install `install_runtime_typechecking()` before test discovery



Jaxtyping import hook instruments subsequently imported `gigl` and `examples` modules



Custom Typeguard adapter retains only annotations containing a Jaxtyping array type



Contracted call: arguments checked before execution; return checked afterward; uncaught `TypeError` fails the test

The custom adapter is deliberate. A vanilla whole-package Typeguard hook would also try to runtime-check GiGL's static-only Protocol and TypeVar annotations, which can be invalid for `isinstance`. The adapter creates a private shape-only view for Typeguard, then preserves the public annotations.

Where the PR adds contracts

Boundary	Examples in PR #748
Loaders and samplers	Node IDs, ABLP seed and label matrices, loader inputs, PPR degrees.
Public models and decoders	<code>forward/decode</code> embeddings and score matrices, including common embedding dimensions.
Losses and task results	Score, ID, and embedding containers. Empty-path tensors were reshaped to retain meaningful matrix rank.
Examples	Link-prediction and node-classification public model methods follow the same contract language.

Policy and tradeoffs

- Check stable tensor boundaries only: loader or sampler inputs, public model `forward/decode`, loss interfaces, and result containers.
- Use exact dtypes only when guaranteed. `Float` preserves mixed-precision compatibility; `Int64/Float32` express hard contracts where appropriate.
- Leave internal tensor operations, dynamic PyG/TorchRec keyed containers, and low-level message passing uncontracted unless they expose a stable, useful boundary.
- The initial scope is intentionally conservative. Coverage increases through exercised tests, while annotations remain easy to remove or revise when an API changes.